SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

NAME: Shaik Azeem

REG.NO: 192472307

COURSE CODE: CSA0712

COURSE NAME: COMPUTER NETWORK

Question 1 — TCP Client–Server Application :

TCP is connection-oriented and reliable. The client connects to the server, sends messages, receives responses, and continues until `"exit"` is entered.

1. TCP Server

server.py > ...

```python
import socket

HOST = "127.0.0.1"
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print("Server started...")
print(f"Listening on {HOST}:{PORT}")

conn, addr = server.accept()
print("Connected by:", addr)

while True:
    data = conn.recv(1024).decode()

    if not data:
        break

    print("Client:", data)

    if data.lower() == "exit":
        conn.send("Connection closed.".encode())
        break

    response = "Server received: " + data
    conn.send(response.encode())

conn.close()
server.close()
print("Server stopped.")
```

2. TCP Client

client.py > ...

```python
import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

print("Connected to server.")
print("Type 'exit' to terminate.")

while True:
    message = input("You: ")

    client.send(message.encode())

    response = client.recv(1024).decode()
    print("Server:", response)

    if message.lower() == "exit":
        break

client.close()
print("Connection closed.")
```

OUTPUT:

```
C:\Users\shaik\Downloads\program\python\test>py client.py
Connected to server.
Type 'exit' to terminate.
You: hello
Server: Server received: hello
You: how are you!
Server: Server received: how are you!
You: Network programming
Server: Server received: Network programming
You: exit
Server: Connection closed.
Connection closed.
```

```
PS C:\Users\shaik\Downloads\program\python\test> py server.py
Server started...
Listening on 127.0.0.1:5000
Connected by: ('127.0.0.1', 54810)
Client: hello
Client: how are you!
Client: Network programming
Client: exit
Server stopped.
```

Question 2 — Reliable Data Transfer over UDP :

UDP itself does **not** guarantee delivery, ordering, or duplicate protection.

So we implement reliability ourselves using:

- Sequence numbers
- ACKs
- Timeout
- Retransmission
- Duplicate detection
- Packet ordering
- Message reassembly

This example intentionally simulates packet loss so that retransmission can be demonstrated.

1. UDP Receiver :

receiver.py > ...

```python
import socket
import random

HOST = "127.0.0.1"
PORT = 6000

receiver = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
receiver.bind((HOST, PORT))

print(f"Receiver listening on {HOST}:{PORT}")

packets = {}

while True:
    data, addr = receiver.recvfrom(1024)

    message = data.decode()

    if message == "END":
        print("\nAll packets received.")
        break

    seq, content = message.split("|", 1)
    seq = int(seq)

    # Simulate packet loss
    if random.random() < 0.2:
        print(f"Packet {seq} lost.")
        continue

    if seq in packets:
        print(f"Duplicate packet {seq} detected.")
    else:
        packets[seq] = content
        print(f"Received packet {seq}: {content}")

    # Send ACK
    receiver.sendto(f"ACK|{seq}".encode(), addr)

# Reassemble message
print("\nReassembled message:")

for seq in sorted(packets):
    print(packets[seq], end=" ")

print()

receiver.close()
```

2. UDP Sender :

sender.py > ...

```python
import socket

SERVER = "127.0.0.1"
PORT = 6000

sender = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sender.settimeout(2)

message = input("Enter message: ")

# Divide message into packets
packet_size = 5
packets = [
    message[i:i + packet_size]
    for i in range(0, len(message), packet_size)
]

for seq, data in enumerate(packets):
    packet = f"{seq}|{data}".encode()

    while True:
        try:
            print(f"Sending packet {seq}: {data}")

            sender.sendto(packet, (SERVER, PORT))

            ack, addr = sender.recvfrom(1024)
            ack = ack.decode()

            ack_type, ack_seq = ack.split("|")

            if ack_type == "ACK" and int(ack_seq) == seq:
                print(f"ACK received for packet {seq}\n")
                break

        except socket.timeout:
            print(f"Timeout! Retransmitting packet {seq}...\n")
```

```python

# Tell receiver transmission is complete
sender.sendto(b"END", (SERVER, PORT))

print("All packets transmitted successfully.")

sender.close()
```

OUTPUT:

```
C:\Users\shaik\Downloads\program\python\test>py sender.py
Enter message: Hello Reliable Data Transfer
Sending packet 0: Hello
Timeout! Retransmitting packet 0...

Sending packet 0: Hello
Timeout! Retransmitting packet 0...

Sending packet 0: Hello
ACK received for packet 0

Sending packet 1:  Reli
ACK received for packet 1

Sending packet 2: able
ACK received for packet 2

Sending packet 3: Data
ACK received for packet 3

Sending packet 4: Trans
ACK received for packet 4

Sending packet 5: fer
ACK received for packet 5

All packets transmitted successfully.
```

```
PS C:\Users\shaik\Downloads\program\python\test> py receiver.py
Receiver listening on 127.0.0.1:6000
Packet 0 lost.
Packet 0 lost.
Received packet 0: Hello
Received packet 1:  Reli
Received packet 2: able
Received packet 3: Data
Received packet 4: Trans
Received packet 5: fer

All packets received.

Reassembled message:
Hello  Reli able  Data  Trans fer
```